Task 1: Order List – Analysis

In this task, I implemented a singly linked list to manage restaurant orders. Each order contains an order ID, customer name, table number, item count, and order time. The OrderList class provides methods to add, remove, search, and display orders.

The time complexity of adding an order is O(n) because the program traverses the list to find the last node before inserting the new order. Removing an order also has O(n) time complexity because the program may need to search through the list to find the specified order ID. Similarly, finding an order takes O(n) time in the worst case because the program checks nodes one by one until it finds the requested order or reaches the end of the list.

A linked list is useful for restaurant order management because the number of orders can change frequently. New orders can be added, and existing orders can be removed without shifting elements as in an array. Each node stores an order and a reference to the next node, allowing the list to grow dynamically.

To test the implementation, I added five orders, removed order 103, searched for order 104, and displayed the final list. The output showed that order 103 was successfully removed, order 104 was found, and four orders remained in the list.

This task helped me understand how linked lists work and how their time complexity affects basic data management operations.

Task 2: Food Request Queue – Analysis

In this task, I implemented a queue using a linked list to manage food preparation requests. The queue follows the First-In, First-Out (FIFO) principle. This means that the first request added to the queue is the first one processed. The FoodQueue class includes enqueue, dequeue, size, and printQueue methods.

The enqueue operation takes O(1) time because a new request is added directly at the rear of the queue. The dequeue operation also takes O(1) time because the request is removed directly from the front. The size operation takes O(1) time because the queue maintains a counter that is updated whenever a request is added or removed. Printing the entire queue takes O(n) time because every request must be visited.

A queue is suitable for food preparation because requests should generally be handled in the order they arrive. This makes the process fair and predictable. A stack would be less appropriate because it follows the Last-In, First-Out (LIFO) principle. Using a stack could cause the newest request to be processed before older requests, delaying customers who ordered earlier.

To test the implementation, I added eight food requests and

processed the first three. The program removed requests 201, 202, and 203 in the correct FIFO order. Five requests remained in the queue.

This task demonstrated how a queue can organize tasks that need to be processed in arrival order.

Task 3: Completed Order Stack – Analysis

In this task, I implemented a stack using a linked list to store completed order records. Each record contains an order ID, customer name, and completion time. The CompletedOrderStack class provides push, pop, peek, size, and printStack methods. The stack follows the Last-In, First-Out (LIFO) principle, meaning that the most recently added record is the first one removed.

The push operation takes O(1) time because a new record is inserted at the top of the stack. The pop operation also takes O(1) time because it removes the top record directly. The peek operation takes O(1) time because it accesses the top record without removing it. Printing all records takes O(n) time because every node must be visited.

A stack is useful when the system needs to access the most recently completed orders first. For example, it can support reviewing recent completions or implementing a simple undo history. If a queue were used instead, records would be removed in the order they were added. This would make it less convenient to access the most recent record first.

To test the implementation, I added five completed order records, checked the most recent record using peek, and removed two records using pop. The program correctly displayed order 305 as the top record, removed orders 305 and 304, and left three records in the stack.

This task helped me understand the LIFO principle and how a stack can be implemented efficiently with a linked list.

Task 4: Restaurant System Integration – Analysis

In this task, I integrated the linked list, queue, and stack into a single Restaurant System. The system manages active orders, food preparation requests, and completed order records. The RestaurantSystem class provides methods to add orders, add food requests, process requests, add completed records, and display the system state.

The linked list stores active orders and allows the system to add, find, and remove orders. The queue manages food preparation requests using the FIFO principle, ensuring that earlier requests are processed first. The stack stores completed order records using the

LIFO principle, making the most recently added record accessible first. Combining these data structures allows each part of the restaurant system to use an appropriate method of organizing data.

I also implemented Bubble Sort to sort active orders by order time. The algorithm compares adjacent elements and swaps them when they are in the wrong order. In this implementation, sorting takes $O(n^2)$ time in the average and worst cases, where n is the number of active orders. The sorting method operates on an array created from the linked list, so it displays the sorted results without changing the original linked list order.

To test the integrated system, I added ten orders and five food requests. I then processed four requests, added two completed order records, and displayed the remaining active orders sorted by order time. The output showed six active orders, one remaining food request, and two completed records.

This task demonstrated how multiple data structures and a sorting algorithm can work together in one application. It also helped me understand how to organize different types of data according to their purposes.

Project Information

Project Name: Restaurant Order Management System

Programming Language: Java

Data Structures: Singly Linked List, Queue, Stack

Sorting Algorithm: Bubble Sort

GitHub Repository: https://github.com/ceydanisadogan/-RestaurantOrderManagement.git

Program Test Outputs

```
TASK 1: ORDER LIST

Initial orders:
Order ID: 101
Customer: Ali
Table: 1
Number of items: 2
Order time: 1000
--------------------
Order ID: 102
Customer: Ayse
Table: 2
Number of items: 3
Order time: 2000
--------------------
```

```
Order ID: 103
Customer: Mehmet
Table: 3
Number of items: 1
Order time: 3000
--------------------
Order ID: 104
Customer: Zeynep
Table: 4
Number of items: 4
Order time: 4000
--------------------
Order ID: 105
Customer: Can
Table: 5
Number of items: 2
Order time: 5000
--------------------

Removing order 103:
Order removed: true

Searching for order 104:
Order ID: 104
Customer: Zeynep
Table: 4
Number of items: 4
Order time: 4000

Final order list:
Order ID: 101
Customer: Ali
Table: 1
Number of items: 2
Order time: 1000
--------------------
Order ID: 102
Customer: Ayse
Table: 2
Number of items: 3
Order time: 2000
--------------------
Order ID: 104
Customer: Zeynep
Table: 4
Number of items: 4
Order time: 4000
--------------------
Order ID: 105
Customer: Can
Table: 5
Number of items: 2
Order time: 5000
--------------------
```

```
Total active orders: 4


TASK 2: FOOD REQUEST QUEUE

Initial food request queue:
Order ID: 201
Food Request Count: 2
Request Time: 1000
--------------------
Order ID: 202
Food Request Count: 1
Request Time: 2000
--------------------
Order ID: 203
Food Request Count: 3
Request Time: 3000
--------------------
Order ID: 204
Food Request Count: 2
Request Time: 4000
--------------------
Order ID: 205
Food Request Count: 1
Request Time: 5000
--------------------
Order ID: 206
Food Request Count: 4
Request Time: 6000
--------------------
Order ID: 207
Food Request Count: 2
Request Time: 7000
--------------------
Order ID: 208
Food Request Count: 1
Request Time: 8000
--------------------

Processing first 3 food requests:
Processed request:
Order ID: 201
Food Request Count: 2
Request Time: 1000
--------------------
Processed request:
Order ID: 202
Food Request Count: 1
Request Time: 2000
--------------------
Processed request:
Order ID: 203
Food Request Count: 3
Request Time: 3000
```

```
--------------------

Remaining food request queue:
Order ID: 204
Food Request Count: 2
Request Time: 4000
--------------------
Order ID: 205
Food Request Count: 1
Request Time: 5000
--------------------
Order ID: 206
Food Request Count: 4
Request Time: 6000
--------------------
Order ID: 207
Food Request Count: 2
Request Time: 7000
--------------------
Order ID: 208
Food Request Count: 1
Request Time: 8000
--------------------
Remaining requests: 5


TASK 3: COMPLETED ORDER STACK

Initial completed order stack:
Order ID: 305
Customer: Can
Completion Time: 13000
--------------------
Order ID: 304
Customer: Zeynep
Completion Time: 12000
--------------------
Order ID: 303
Customer: Mehmet
Completion Time: 11000
--------------------
Order ID: 302
Customer: Ayse
Completion Time: 10000
--------------------
Order ID: 301
Customer: Ali
Completion Time: 9000
--------------------

Most recent completed order (peek):
Order ID: 305
Customer: Can
Completion Time: 13000
```

```
Removing 2 records (pop):
Popped record:
Order ID: 305
Customer: Can
Completion Time: 13000
--------------------
Popped record:
Order ID: 304
Customer: Zeynep
Completion Time: 12000
--------------------

Remaining completed order stack:
Order ID: 303
Customer: Mehmet
Completion Time: 11000
--------------------
Order ID: 302
Customer: Ayse
Completion Time: 10000
--------------------
Order ID: 301
Customer: Ali
Completion Time: 9000
--------------------
Remaining records: 3


TASK 4: RESTAURANT SYSTEM INTEGRATION

Active Orders:
Order ID: 401
Customer: Ali
Table: 1
Number of items: 2
Order time: 5000
--------------------
Order ID: 402
Customer: Ayse
Table: 2
Number of items: 3
Order time: 2000
--------------------
Order ID: 403
Customer: Mehmet
Table: 3
Number of items: 1
Order time: 7000
--------------------
Order ID: 404
Customer: Zeynep
Table: 4
Number of items: 4
```

```
Order time: 1000
--------------------
Order ID: 405
Customer: Can
Table: 5
Number of items: 2
Order time: 6000
--------------------
Order ID: 406
Customer: Elif
Table: 6
Number of items: 3
Order time: 3000
--------------------
Order ID: 407
Customer: Ahmet
Table: 7
Number of items: 1
Order time: 9000
--------------------
Order ID: 408
Customer: Ece
Table: 8
Number of items: 2
Order time: 4000
--------------------
Order ID: 409
Customer: Mert
Table: 9
Number of items: 4
Order time: 8000
--------------------
Order ID: 410
Customer: Selin
Table: 10
Number of items: 1
Order time: 1500
--------------------

Food Preparation Queue:
Order ID: 401
Food Request Count: 2
Request Time: 1000
--------------------
Order ID: 402
Food Request Count: 1
Request Time: 2000
--------------------
Order ID: 403
Food Request Count: 3
Request Time: 3000
--------------------
Order ID: 404
Food Request Count: 2
```

```
Request Time: 4000
--------------------
Order ID: 405
Food Request Count: 1
Request Time: 5000
--------------------

Processing 4 food requests:
Order ID: 401
Food Request Count: 2
Request Time: 1000
--------------------
Order ID: 402
Food Request Count: 1
Request Time: 2000
--------------------
Order ID: 403
Food Request Count: 3
Request Time: 3000
--------------------
Order ID: 404
Food Request Count: 2
Request Time: 4000
--------------------

Remaining food requests:

Food Preparation Queue:
Order ID: 405
Food Request Count: 1
Request Time: 5000
--------------------
Remaining requests: 1

Completed Orders:
Order ID: 402
Customer: Ayse
Completion Time: 12000
--------------------
Order ID: 401
Customer: Ali
Completion Time: 10000
--------------------
Completed records: 2

Active orders sorted by waiting time (oldest first):
Order ID: 410
Customer: Selin
Table: 10
Number of items: 1
Order time: 1500
--------------------
Order ID: 406
Customer: Elif
```

```
Table: 6
Number of items: 3
Order time: 3000
--------------------
Order ID: 408
Customer: Ece
Table: 8
Number of items: 2
Order time: 4000
--------------------
Order ID: 405
Customer: Can
Table: 5
Number of items: 2
Order time: 6000
--------------------
Order ID: 409
Customer: Mert
Table: 9
Number of items: 4
Order time: 8000
--------------------
Order ID: 407
Customer: Ahmet
Table: 7
Number of items: 1
Order time: 9000
--------------------

Total integrated orders: 6
Integration completed successfully.
```